

How would you scale this for 500 concurrent users running reports?

I would start by increasing the database connection pool size so the application can handle more concurrent requests efficiently.

Since report generation is primarily read-heavy, I would add database read replicas and route reporting queries to them instead of the primary database. This reduces the load on the primary database and improves overall throughput.

A primary-replica architecture would work well here: the primary database handles all write operations, such as creating employees, goals, and performance reviews, while replicas serve read operations like summaries, reports, and review history.

If traffic continues to grow, I would run multiple instances of the application behind a load balancer. Because the application is stateless, horizontal scaling is straightforward.

What would you do if `GET /cycles/{id}/summary` becomes slow with 100k+ reviews?

The first step would be to analyze the query execution plan using `EXPLAIN ANALYZE` and identify any bottlenecks. `EXPLAIN ANALYZE` shows whether indexes are being used, how many rows are being scanned, and how much time each step of the query is taking. This helps pinpoint the exact cause of the slowdown.

In many cases, performance issues can be resolved by adding or optimizing indexes. For this use case, indexing columns involved in filtering and aggregation, such as `cycle_id`, would be important.

If aggregating large amounts of data still becomes expensive, I would introduce a pre-computed summary table. Whenever a review is created or updated, the summary data will be refreshed. The API would then read directly from the summary table instead of recalculating aggregates across 100k+ reviews on every request. This significantly reduces query execution time and database load.

Where would you add caching?

I believe in this, that we should only cache endpoints where the cache key is predictable

I would cache `GET /cycles/{id}/summary` because it is likely to be one of the most frequently accessed and computationally expensive endpoints.

A distributed cache such as Redis with a short TTL (for example, 5–10 minutes) would reduce database load significantly. When new reviews are submitted for a cycle, the corresponding cache entry can be invalidated.

I would also consider caching `GET /employees/{id}/reviews`, since review history generally changes infrequently.

I wouldn't cache `GET /employees?department=&minRating=` because the parameters can have too many combinations.

Think about it — department alone can be Engineering, Product, HR, Finance, and minRating can be 1.0, 1.5, 2.0, 2.5... all the way to 5.0. Every unique combination becomes a separate cache entry in Redis.

This blows up your Redis memory fast, and when an employee's rating changes, we have no idea which of these 100 cache keys to invalidate. If we delete all of them? That will defeat the purpose of caching.

Assumptions

- Multiple managers can review the same employee during a review cycle.
- A reviewer can submit only one review for a given employee within a cycle, enforced through a unique constraint.
- `reviewerId` is currently stored as a string identifier; in a production system it would likely reference a dedicated Users table.
- H2 is used only for demonstration purposes. Migrating to a production database such as PostgreSQL would mainly involve configuration changes and database-specific migrations.
- Goals are tracked at the employee and review-cycle level and are independent of individual performance reviews.